# Generative AI and Its Applications

## Unit 3

(UE23CS342BA4)

## Fine-Tuning, Data Resolution/Precision, Quantization, MoE

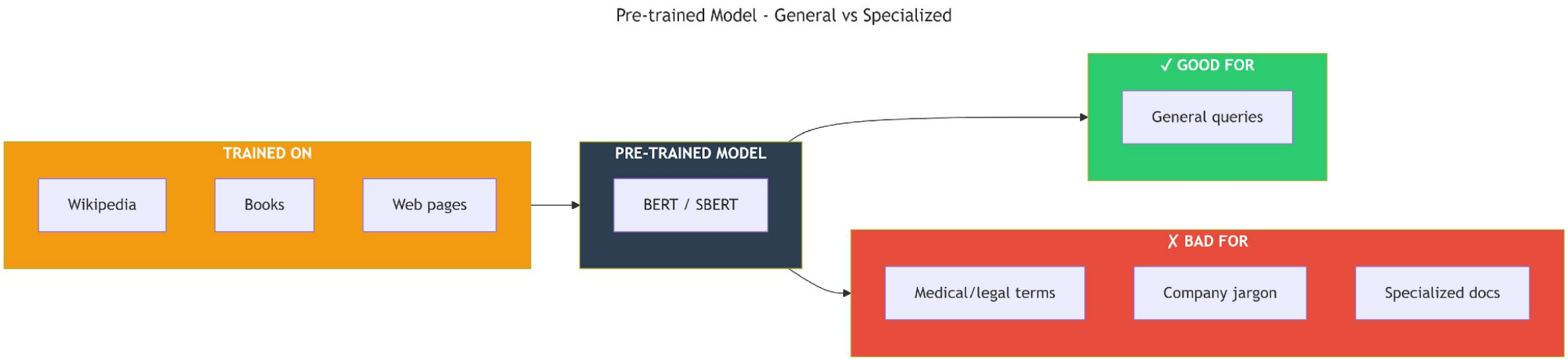
**Dr. Pooja Agarwal**
Dept. of CSE, PES University, Bangalore

## Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

# Why Fine-Tune for RAG?!

**The Problem with Pre-trained Models:**

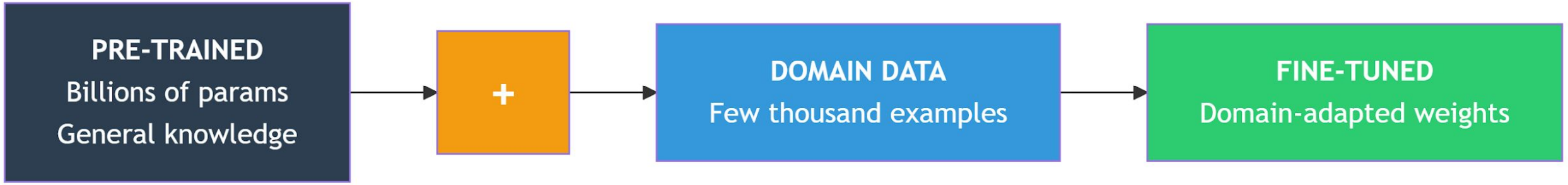



A pre-trained model is like a **medical school graduate** – they know general medicine but need **residency training** to become a specialist surgeon. Fine-tuning is that specialized training for your specific domain.

# Why Fine-Tune for RAG?!

**What is Fine-Tuning? :** **Fine-Tuning** is the process of taking a pre-trained model and further training it on a smaller, domain-specific dataset to adapt its knowledge and representations to a particular task or domain.

Fine-Tuning for Domain Adaptation




Loss Before Fine-tuning: —— high ——▶
Loss After Fine-tuning: —— low ——▶

The model updates weights to minimize loss on your specific data while retaining general knowledge.

# Why Fine-Tune for RAG?!

**RAG systems have three fine-tunable components::**

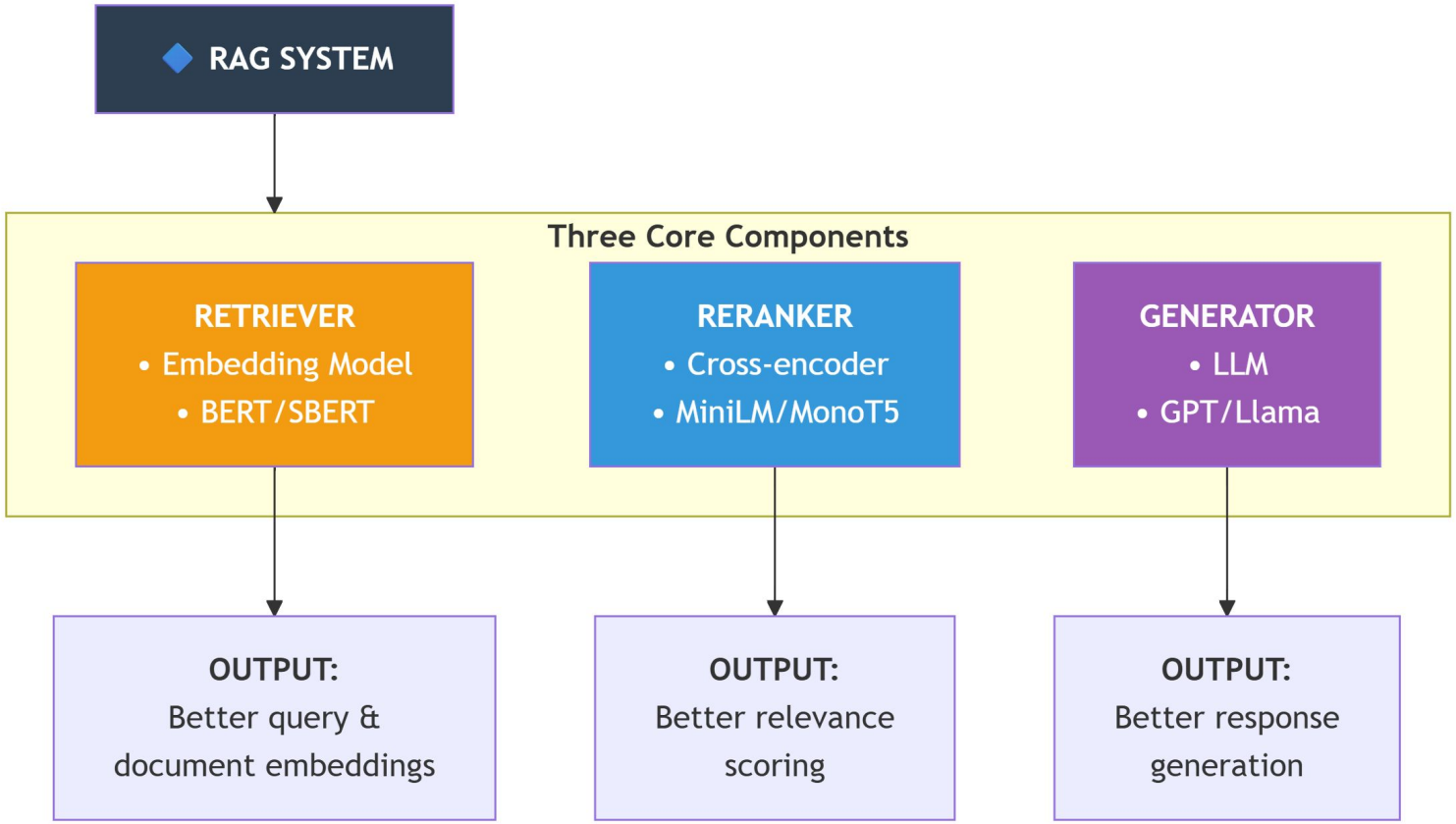
**Which to Fine-Tune?**

- **Retriever:** When your documents have domain-specific terminology
- **Reranker:** When you need high-precision relevance judgments
- **Generator:** When responses need specific style/format/domain knowledge

## FINE-TUNING FOR RAG

# Why Fine-Tune for RAG?!

**Fine-Tuning the Retriever (Embedding Model):**



# Why Fine-Tune for RAG?!

**Fine-Tuning the Retriever (Embedding Model):**

**Goal:** Make embeddings place similar domain documents closer together.

**Training Data Format:**

Triplets: (Query, Positive Document, Negative Document)

Example (Medical Domain):
Query: "What are the side effects of Metformin?"
Positive: "Metformin may cause gastrointestinal disturbances including diarrhea..."
Negative: "Lifestyle modifications for diabetes management..."

# Why Fine-Tune for RAG?!

**Loss Function: Multiple Negative Ranking Loss (MNRL):**

$$\text{Loss} = -\log\left( e^{\text{sim}(q,d^+)/\tau} \,/\, \Sigma\, e^{\text{sim}(q,d\square)/\tau} \right)$$

Where:
- sim(q,d) = cosine similarity
- τ = temperature parameter (typically 0.01-0.1)
- denominator includes 1 positive + N negatives

# Numerical Example:

**Initial Model Prediction**

Suppose a pretrained model predicts the probability for a binary classification task.

Input example: **"This movie is great"**

True label: **1 (Positive)**

Model prediction before fine-tuning:

$$\hat{y} = 0.40$$

Loss function: **Binary Cross-Entropy**

$$\boldsymbol{Loss = -\left(y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right)}$$

# Numerical Example:

**Step 1: Loss Before Fine-Tuning**

$$y = 1, \hat{y} = 0.40$$
$$Loss = -\left(1 \cdot \log(0.40)\right)$$
$$Loss = -(-0.916)$$
$$Loss \approx 0.916$$

Loss is **high**, meaning the prediction is far from the true label.

**Step 2: After Fine-Tuning**

After training on domain-specific data, the model updates its weights.

New prediction:

$\hat{y} = 0.90$

**Step 3: Loss After Fine-Tuning**

$$Loss = -\left(1 \cdot \log(0.90)\right)$$
$$Loss = -(-0.105)$$
$$Loss \approx 0.105$$

| Stage | Prediction | Loss |
|---|---|---|
| Before Fine-Tuning | 0.40 | 0.916 |
| After Fine-Tuning | 0.90 | 0.105 |

Loss Before Fine-tuning: 0.916

Loss After Fine-tuning: 0.105

## FINE-TUNING FOR RAG

# Why Fine-Tune for RAG?!

**Fine-Tuning the Retriever - Step by Step:**

**Step 1: Prepare Training Data**

```
# Create query-positive-negative triplets
training_data = [
    {
        "query": "machine learning algorithms",
        "positive": "Supervised and unsupervised learning
methods...",
        "negatives": [
            "History of ancient Rome",
            "Climate change effects",
            "Quantum physics basics"
        ]
    },
]
```

**Step 2: Choose Base Model**

```
from sentence_transformers import SentenceTransformer,
losses
from torch.utils.data import DataLoader

# Load pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')
```

# Why Fine-Tune for RAG?!

**Step 3: Create Dataset and DataLoader**

```
train_examples = []
for item in training_data:
    # Add positive pair
    train_examples.append((item['query'], item['positive'], 1.0))
    # Add negative pairs
    for neg in item['negatives']:
        train_examples.append((item['query'], neg, 0.0))

train_dataloader = DataLoader(train_examples, shuffle=True,
batch_size=16)
```

**Step 4: Define Loss and Train**

```
# Use MultipleNegativesRankingLoss
train_loss = losses.MultipleNegativesRankingLoss(model)

# Fine-tune
model.fit(
    train_objectives=[(train_dataloader, train_loss)],
    epochs=3,
    warmup_steps=100,
    output_path='./fine_tuned_retriever'
)
```

# Fine-Tuning the Reranker (Cross-Encoder)

**Goal:** Make cross-encoder better at judging relevance for your domain.

## Training Data Format

**Training Data Format**

Models are trained on **query–document pairs with relevance scores**.

Example:

Query: **“diabetes treatment guidelines”**

Document: **“ADA 2023 guidelines recommend Metformin as first-line therapy”**

Label: **5 (highly relevant)**

Query: **“diabetes treatment guidelines”**

Document: **“History of insulin discovery”**

Label: **1 (barely relevant)**

# Fine-Tuning the Reranker (Cross-Encoder)

**Goal:** Make cross-encoder better at judging relevance for your domain.

**Loss Function: Binary Cross-Entropy or MSE:**

**Binary Classification (Relevant / Not Relevant)**

Binary Cross-Entropy Loss:

$$Loss = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Where:

- **y** = true label (0 or 1)
- **ŷ** = predicted probability

**Score Regression (Relevance Scores)**

Mean Squared Error:

$$Loss = \left(y - \hat{y}\right)^2$$

Used when labels are **continuous scores (0–5)**.

# Numerical Example (Binary Cross-Entropy):

True label:

$y = 1$ (relevant)

Model prediction:

$\hat{y} = 0.85$

Loss calculation:

$$\begin{aligned} \text{Loss} &= -[1 \cdot \log(0.85) + 0 \cdot \log(0.15)] \\ &= -\log(0.85) \\ &= -(-0.1625) \\ &= 0.1625 \end{aligned}$$

Loss is **low**, meaning the prediction is close to the correct label.

**Example with Poor Prediction**

If the model predicts:

$\hat{y} = 0.40$

Loss becomes:

$$\begin{aligned} \text{Loss} &= -\log(0.40) \\ &= 0.916 \end{aligned}$$

| Prediction | Loss | Interpretation |
|---|---|---|
| 0.85 | 0.1625 | Good prediction |
| 0.40 | 0.916 | Poor prediction |

- Lower loss means **better model predictions**.
- Training updates model weights to **minimize this loss over all query–document pairs**.

# Numerical Example (Binary Cross-Entropy):

**Fine-Tuning the Reranker – Implementation:**

https://huggingface.co/blog/train-reranker#:~:text=Through%20finetuning%2C%20the%20model%20can,those%20baselines%20are%20much%20bigger.

# Fine-Tuning the Generator (LLM) Instruction Fine-Tuning and PEFT

**Goal**

Make a large language model generate responses in a **desired style, format, and domain knowledge**.

**Training Data Format:**

Instruction Fine-Tuning uses:
Instruction + Context + Response

Example (Medical QA):
Instruction:
Answer the medical question based on the context.
Context:
Metformin is contraindicated in patients with eGFR < 30 mL/min.
Response:
Metformin should not be used in patients with severe kidney disease (eGFR <30 mL/min) due to the risk of lactic acidosis.

# Fine-Tuning the Generator (LLM) Instruction Fine-Tuning and PEFT

**Goal**

Make a large language model generate responses in a **desired style, format, and domain knowledge**.

**Parameter-Efficient Fine-Tuning (PEFT):**

Full Fine-Tuning

Update **all model parameters**.

Example:

Llama2 7B → 7 billion parameters must be updated.

This requires **large GPU memory and high cost**.

**LoRA (Low-Rank Adaptation)**

Pretrained weights are **frozen** and only small adapter layers are trained.

Pretrained Weight Matrix:

$$W \in \mathbb{R}^{dxk} \text{ (frozen)}$$

LoRA Update

$$\Delta W = B \cdot A$$

Where

$$A \in \mathbb{R}^{rxk}$$

$$B \in \mathbb{R}^{dxr}$$

$$r << \min(d,k)$$

Typical rank r = 8–64

Trainable parameters are **less than 1% of the original model**.

# LoRA – Detailed Explanation

**How LoRA Works**

Original Layer:
Input x
⟶ Weight W
⟶ Output y

LoRA Modified Layer:
Input x
⟶ W · x (frozen)
⟶ B · A · x (trainable)
Final Output

$$y = W \cdot x + B \cdot A \cdot x$$

**Matrix Definitions**

W : Original pretrained weights (d × k) – Frozen
A : Low-rank matrix (r × k) – Trainable
B : Low-rank matrix (d × r) – Trainable
r : Rank (8–64), much smaller than d or k

**Parameter Savings Example:**

For a transformer layer:
d = 4096
k = 4096
Full Fine-Tuning
4096 × 4096 = 16.7 million trainable parameters

LoRA with rank r = 8
A parameters = 8 × 4096 = 32,768
B parameters = 4096 × 8 = 32,768
Total trainable parameters = 65,536
This is **0.39% of the original parameters**.

# LoRA Numerical Example

Input vector
x = [1.0, 2.0]
Original weight matrix
W =[0.5 0.3
0.2 0.4]

**Step 1: Original Forward Pass**

**W · x**

= [0.5×1.0 + 0.3×2.0,0.2×1.0 + 0.4×2.0]
= [1.1, 1.0]

**Step 2: LoRA Contribution**

LoRA matrices (rank r = 1)
A = [0.1 0.2]
B =[0.3 0.4]
Compute: **A · x**
= [0.1×1.0 + 0.2×2.0]
= [0.5]
Then: **B · (A · x)**
= [0.3×0.5, 0.4×0.5]
= [0.15, 0.20]

**Step 3: Combined Output**

**y = W·x + B·A·x**

y = [1.1, 1.0] + [0.15, 0.20]
y = [1.25, 1.20]

**During Training**

W remains **fixed**
W =[0.5 0.3
0.2 0.4]

- Only **A and B matrices update using gradients**.
- This allows efficient fine-tuning with **very few trainable parameters**.

# LoRA Numerical Example

**Fine-Tuning an LLM for Command Generation**

https://huggingface.co/docs/transformers/en/training

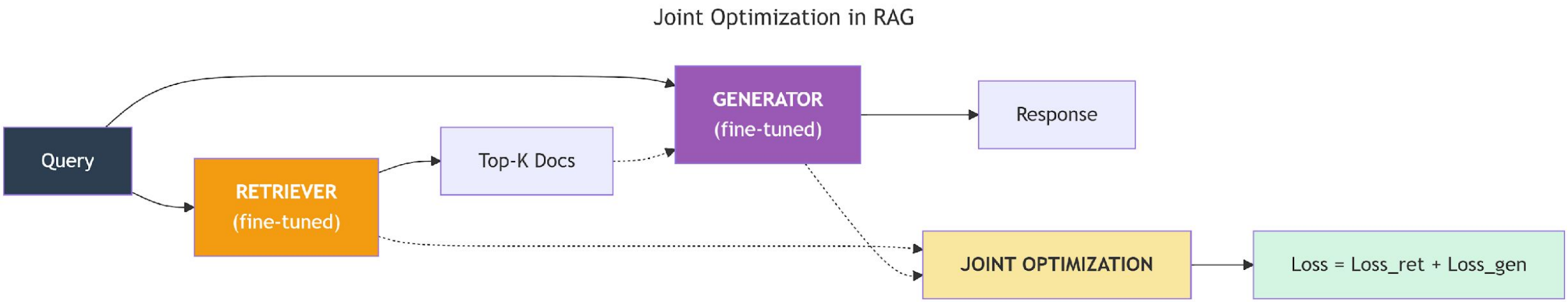
# RA-DIT - Retrieval-Augmented Dual Instruction Tuning

**State-of-the-art: Fine-tune both retriever and generator together**

Fine-tune **both the retriever and the generator together** for better retrieval-augmented generation.

This approach improves:

- Document retrieval quality
- Response generation accuracy
- End-to-end RAG system performance



# RA-DIT - Retrieval-Augmented Dual Instruction Tuning

**Training Objective:**

Joint loss combines **retrieval loss** and **generation loss**.

$$L_total = \alpha \cdot L_retriever + \beta \cdot L_generator$$

Where:

- **L_retriever** → improves document retrieval
- **L_generator** → improves response generation
- **α, β** → weighting parameters

Example:

$\alpha = 0.5$

$\beta = 0.5$

# RA-DIT – Numerical Example

**Step 1: Retriever Loss** (Multiple Negatives Ranking Loss)
Query: "diabetes treatment“

Positive document: "Metformin is first-line for type 2 diabetes"
Negative documents: History of insulin, Dietary guidelines, Exercise benefits

Similarity scores:

| Pair | Similarity |
|---|---|
| $sim(q,d^{+})$ | 0.92 |
| $sim(q,d^{-1})$ | 0.45 |
| $sim(q,d^{-2})$ | 0.38 |
| $sim(q,d^{-3})$ | 0.41 |

Temperature:
$\tau = 0.05$
Retriever loss:

```
L_ret = -log( e^(0.92/0.05) /
              (e^(0.92/0.05) + e^(0.45/0.05)
               + e^(0.38/0.05) + e^(0.41/0.05)))
= -log( e^18.4 /(e^18.4 + e^9 + e^7.6 + e^8.2))
                  ≈ -log(0.9998)
                     ≈ 0.0002
```

Retriever loss is **very small**, meaning the model correctly ranks the relevant document highest.

# RA-DIT – Numerical Example Generator Loss

**Step 2: Generator Loss**
(Cross-Entropy)
Target response:
`"Metformin is prescribed as first-line treatment..."`
Predicted logits for first token:

| Token | Logit |
|---|---|
| Metformin (correct) | 5.2 |
| Insulin | 2.1 |
| Diet | 1.8 |

Softmax probability:

```
P(target) = e^5.2/(e^5.2 + e^2.1 + e^1.8 +...)
          = 181.3/(181.3 + 8.2 + 6.0 +...)
                                ≈ 0.92
```

Generator loss:

```
L_gen = -log(0.92)
        ≈ 0.083
```

# RA-DIT – Numerical Example

**Step 3: Joint Loss**

Weights:

$\alpha = 0.5$

$\beta = 0.5$

$$L_total = \alpha \cdot L_ret + \beta \cdot L_gen$$

$L_total = 0.5 \times 0.0002 + 0.5 \times 0.083$

$L_total \approx 0.0416$

RA-DIT jointly optimizes:

- **Retriever accuracy**
- **Generator response quality**

This leads to **better retrieval-augmented LLM systems**.

**Rule of Thumb**

Fine-tune the component **closest to the specific requirement**:

1. Start with the **retriever** to improve document retrieval.
2. Fine-tune the **reranker** if higher ranking precision is needed.
3. Fine-tune the **generator** only when specific response styles, formats, or reasoning behaviors are required.

# When to Fine-Tune – Decision Matrix

| Scenario | Retriever | Reranker | Generator |
|---|---|---|---|
| **Domain-specific vocabulary (medical, legal, technical)** | Required | Helpful | Optional |
| **Specific response format (JSON, structured tables, templates)** | Not required | Not required | Required |
| **High precision retrieval required** | Optional | Required | Not required |
| **Low-resource domain with limited training data** | Helpful | Helpful | Optional |
| **Multilingual applications** | Required | Helpful | Helpful |
| **Company internal documents or proprietary knowledge** | Required | Helpful | Optional |

# WHAT IS DATA RESOLUTION & PRECISION?

**Definition**

**Data resolution (precision)** refers to the numerical accuracy used to represent **model weights, activations, and embeddings**.

Higher precision provides more accurate numerical representation but requires **more memory and computation**.

Common Numeric Precisions:

| Precision | Bits | Numerical Range | Typical Use |
|---|---|---|---|
| FP32 | 32 | ±1.4e-45 to 3.4e38 | Training (master weights) |
| FP16 | 16 | ±5.96e-8 to 65504 | Training and inference |
| BF16 | 16 | ±1.18e-38 to 3.4e38 | Stable training |
| INT8 | 8 | -128 to 127 | Quantized inference |
| INT4 | 4 | -8 to 7 | Extreme compression |
| Binary | 1 | -1, 0, 1 | Specialized hardware |

# Precision in RAG Components

Where Precision Matters in a RAG Pipeline

| Component | Precision Requirement | Reason |
|---|---|---|
| Embeddings | High precision | Accurate similarity calculations |
| Vector Search | Medium precision | Approximate search acceptable |
| Cross-Encoder (Reranker) | High precision | Accurate ranking scores required |
| LLM Generator | Lower precision acceptable | Quantization often used for efficiency |

Impact of Precision on Performance

| Component | FP32 | FP16 | INT8 | INT4 |
|---|---|---|---|---|
| Embedding similarity accuracy | 100% | 99.8% | 98.5% | 95% |
| Re-ranking accuracy | 100% | 99.7% | 97% | 92% |
| Generation quality | 100% | 99.9% | 99% | 96% |
| Memory usage (7B model) | 28 GB | 14 GB | 7 GB | 3.5 GB |

# Precision in RAG Components

**Floating-Point Precision**

**FP32 Representation**

FP32 format:

[1 sign bit | 8 exponent bits | 23 mantissa bits]

Example number:

3.14159

Components:

Sign = 0

Exponent = 128

Actual exponent = 128 − 127 = 1

Formula:

**Value = (−1)^sign × 2^(exponent − bias) × (1 + mantissa)**

**FP16 Representation**

FP16 format:

[1 sign bit | 5 exponent bits | 10 mantissa bits]

Example conversion:

3.14159 → 3.140625

**Error = 0.00097**

**BF16 Representation**

BF16 format:

[1 sign bit | 8 exponent bits | 7 mantissa bits]

Example conversion:

3.14159 → 3.0

**Error = 0.14159**

BF16 keeps the **same dynamic range as FP32**, which helps prevent overflow during training.

# Precision in RAG Components

**Floating-Point Precision**

Precision Loss Example

| Representation | Stored Value | Error |
|---|---|---|
| FP32 | 0.123456789 | 0 |
| FP16 | 0.123458862 | 0.000002073 |
| BF16 | 0.123046875 | 0.000409914 |
| INT8 (scale 0.01) | 0.12 | 0.003456789 |

# Embedding Precision Example

Original FP32 embeddings:
q = [0.123456, 0.789012]
d = [0.234567, 0.890123]

$\|q\| = \sqrt{(0.123456^2 + 0.789012^2)} = \sqrt{(0.01524 + 0.62254)} = \sqrt{0.63778} = 0.7986$
$\|d\| = \sqrt{(0.234567^2 + 0.890123^2)} = \sqrt{(0.05502 + 0.79232)} = \sqrt{0.84734} = 0.9205$

cosine = (0.02896 + 0.70234) / (0.7986 × 0.9205) = 0.73130 / 0.7351 = 0.9948

INT8 quantized (scale factor 100):
q_int = [12, 79]  (0.123456×100=12.3→12, 0.789012×100=78.9→79)
d_int = [23, 89]

Reconstructed:
q_rec = [0.12, 0.79]
d_rec = [0.23, 0.89]

Cosine similarity = $(0.12\times0.23 + 0.79\times0.89) / (\sqrt{(0.0144+0.6241)} \times \sqrt{(0.0529+0.7921)})$
= (0.0276 + 0.7031) / (√0.6385 × √0.8450)
= 0.7307 / (0.7991 × 0.9192)
= 0.7307 / 0.7346
= 0.9947

Error = |0.97 - 0.9947| = 0.0247 (2.5% error)

## DATA RESOLUTION & PRECISION

# Precision-Aware Retrieval

**High Precision Mode**
Precision: FP32 or FP16
Characteristics:
- Full precision embeddings
- Exact cosine similarity
- Higher memory usage

Best suited for:
- Critical applications
- Smaller document collections

**Medium Precision Mode**
Precision: INT8
Characteristics:
- Quantized vectors
- Approximate similarity via dot product
- Lower memory consumption

Advantages:
- 4× memory reduction
- 2× faster retrieval

Best suited for:
- Large document collections

**Low Precision Mode**
Precision: INT4 or Binary
Characteristics:
- Binary hash representations
- Hamming distance similarity
- Extremely small storage footprint

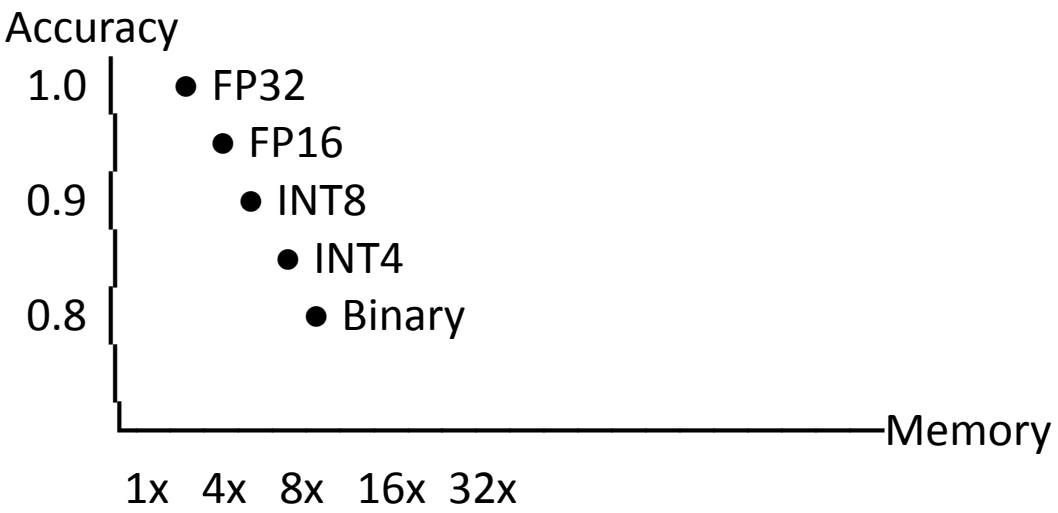
Advantages:
- 32× memory reduction
- Up to 10× faster retrieval

Best suited for:
- Billion-scale datasets
- Recall-focused retrieval

## DATA RESOLUTION & PRECISION

# Precision vs Memory Trade-off

| Precision | Relative Accuracy | Memory Usage |
|---|---|---|
| FP32 | Highest | 1× |
| FP16 | Very high | 2× reduction |
| INT8 | High | 4× reduction |
| INT4 | Moderate | 8–16× reduction |
| Binary | Lowest | 32× reduction |



# QUANTIZATION TECHNIQUES

## QUANTIZATION TECHNIQUES What is Quantization?

**Definition**

**Quantization** is the process of reducing the numerical precision of model parameters and activations from high precision formats (such as FP32) to lower precision formats (such as INT8 or INT4).

Purpose:

- Reduce memory usage
- Accelerate inference
- Enable deployment on limited hardware

**Analogy**

Quantization is similar to rounding a value such as:

3.14159 → 3.14

The number becomes easier to store and compute with, but introduces a small approximation error.

# QUANTIZATION TECHNIQUES

**QUANTIZATION TECHNIQUES** What is Quantization?

FP32 Weights: [0.1234, -0.5678, 1.2345, -2.3456, ...]

↓ ↓ ↓ ↓

INT8 Quantized: [12, -57, 123, -235, ...]

↓ ↓ ↓ ↓

Scale Factor: 0.01 0.01 0.01 0.01

Reconstructed: [0.12, -0.57, 1.23, -2.35, ...]

Error: 0.0034 0.0022 0.0045 0.0044

# QUANTIZATION TECHNIQUES

**Quantization Mathematics** What is Quantization?

**Linear Quantization (Symmetric)**

Quantization formula:

$$x_int = clamp(round(x / s))$$

Where:

s = scale factor

b = number of bits

Scale factor:

$$s = max(|x|) / (2^{(b-1)} - 1)$$

**Dequantization**

$$x_dequant = x_int \times s$$

**Numerical Example:**

FP32 weights:
[0.5, -1.2, 2.3, -0.8, 1.7]

Maximum absolute value:
2.3

INT8 range:
-127 to 127

Scale factor:
s = 2.3 / 127 = 0.01811

# QUANTIZATION TECHNIQUES

## Quantization Mathematics

Quantization

| FP32 | Calculation | INT8 |
|---|---|---|
| 0.5 | 0.5 / 0.01811 | 28 |
| -1.2 | -1.2 / 0.01811 | -66 |
| 2.3 | 2.3 / 0.01811 | 127 |
| -0.8 | -0.8 / 0.01811 | -44 |
| 1.7 | 1.7 / 0.01811 | 94 |

Quantized values:
[28, -66, 127, -44, 94]

# QUANTIZATION TECHNIQUES

## Quantization Mathematics

Dequantization

| INT8 | Dequantized | Error |
|---|---|---|
| 28 | 0.507 | 0.007 |
| -66 | -1.195 | 0.005 |
| 127 | 2.300 | 0 |
| -44 | -0.797 | 0.003 |
| 94 | 1.702 | 0.002 |

# QUANTIZATION TECHNIQUES

**Quantization Methods**

Trained Model (FP32) ⟶ Calibration Data ⟶ Quantized Model (INT8)

└── No training, just calibration

└── Fast, some accuracy loss

**Post-Training Quantization (PTQ)**

Pipeline:

Trained Model (FP32) → Calibration Data → Quantized Model (INT8)

Characteristics:

- No retraining required
- Very fast conversion
- Slight accuracy loss possible

**Quantization-Aware Training (QAT)**

Pipeline:

Training with simulated quantization → Final Quantized Model

Characteristics:

- Quantization simulated during training
- Higher final accuracy
- Requires additional training time

Train with simulated quantization ⟶ Final Quantized Model

└── Higher accuracy, slower training

└── Best accuracy

# QUANTIZATION TECHNIQUES

## Quantization Methods

Accuracy Comparison

| Method | Accuracy | Time | Use Case |
|---|---|---|---|
| FP32 baseline | 100% | Training complete | Reference model |
| PTQ INT8 | 98.5% | Minutes | Rapid deployment |
| QAT INT8 | 99.2% | Hours | Production systems |
| PTQ INT4 | 95% | Minutes | Edge devices |
| QAT INT4 | 97% | Hours | Mobile deployment |

# QUANTIZATION TECHNIQUES

## Quantization in RAG Components

Embedding Model Quantization

```
from sentence_transformers import SentenceTransformer
import torch

model = SentenceTransformer('all-MiniLM-L6-v2')

model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)
```

Performance impact:
Memory: 80 MB → 20 MB
Speed: approximately 1.5× faster
Accuracy: approximately 99% preserved

# QUANTIZATION TECHNIQUES

## Quantization in RAG Components

Cross-Encoder Quantization

```
from transformers import AutoModelForSequenceClassification
import torch

model = AutoModelForSequenceClassification.from_pretrained(
    'cross-encoder/ms-marco-MiniLM-L-6-v2'
)

model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)
```

Performance impact:
Memory: 120 MB → 30 MB
Speed: approximately 2× faster
Accuracy: approximately 98.5% preserved

# QUANTIZATION TECHNIQUES

## Quantization in RAG Components

LLM Quantization (Generator)

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    load_in_4bit=True,
    device_map='auto'
)
```

Performance impact:
Memory: 28 GB → 4 GB
Speed: approximately 3× faster
Quality: 95–98% of FP32 quality

# QUANTIZATION TECHNIQUES

## Quantization Impact in RAG

Example Query:
"What is the capital of France?"

**Step 1: Query Embedding (INT8)**
Original vector:
[0.234, -0.567, 0.891]
Quantized vector:
[23, -57, 89]
Approximate error per dimension:
±0.005

Step 2: Retrieval

| Precision | Recall@5 |
|---|---|
| FP32 | 0.95 |
| INT8 | 0.93 |

Loss: approximately 2%

Step 3: Re-Ranking

| Document | FP32 Score | INT8 Score |
|---|---|---|
| Doc1 | 0.92 | 0.91 |
| Doc2 | 0.89 | 0.88 |
| Doc3 | 0.85 | 0.84 |

Ranking order remains unchanged.

**Step 4: Generation**
FP32 model output:
"The capital of France is Paris."
4-bit model output:
"The capital of France is Paris."
Outputs are identical.

**Final Result**
Overall system quality:
Approximately **98% of FP32 performance**
Memory usage:
Approximately **1/8 of original memory**

# MIXTURE OF EXPERTS (MoE)

**What is Mixture of Experts (MoE)?**

**Definition:**

**Mixture of Experts (MoE)** is a neural network architecture that uses multiple specialized sub-networks ("experts") with a gating mechanism that routes each input to the most appropriate expert(s).

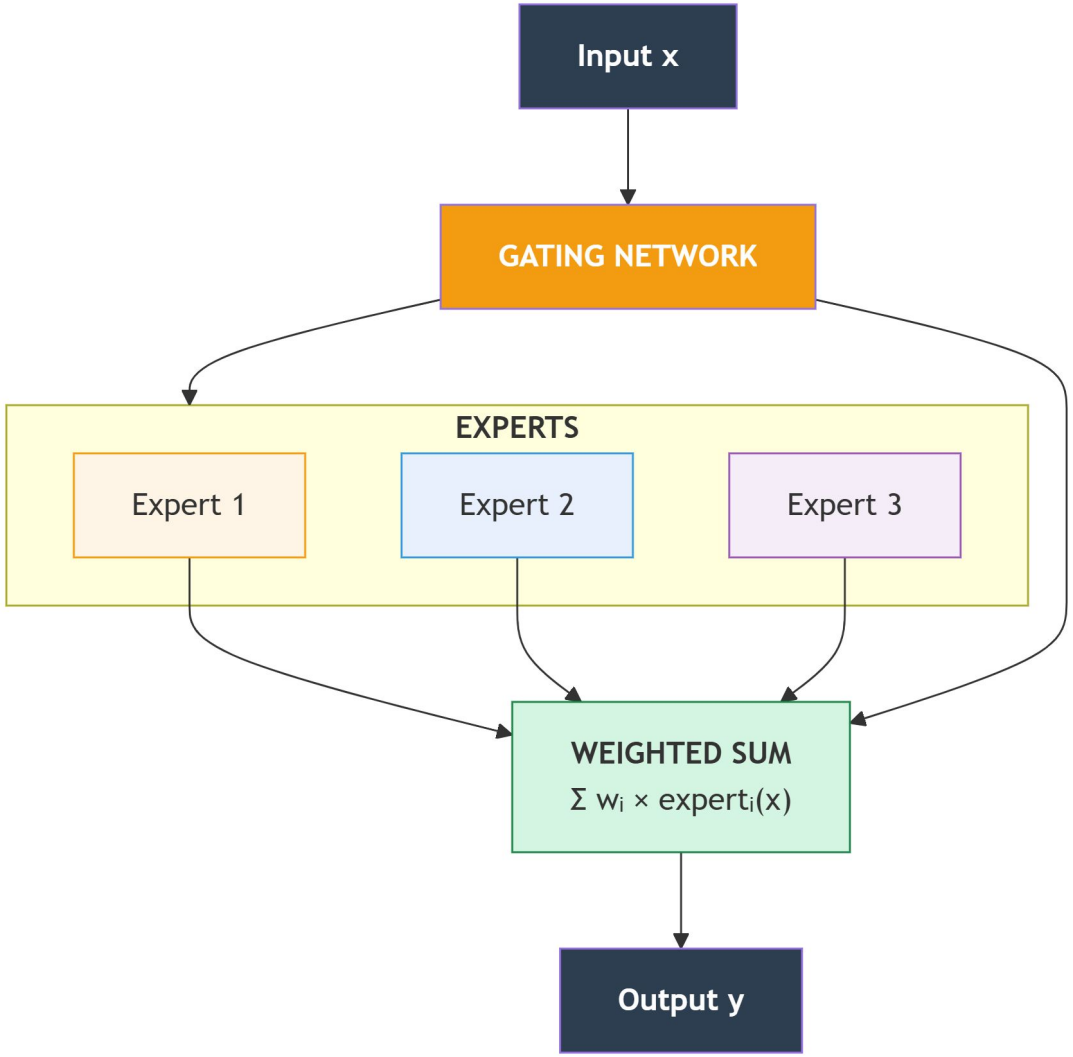
**Analogy:**

MoE is like a **hospital with specialist doctors**:

- General physician (gate) routes patients to specialists
- Cardiologist (expert 1) handles heart issues
- Neurologist (expert 2) handles brain issues
- Orthopedist (expert 3) handles bone issues
- Each patient sees only relevant specialists, not all doctors!

# MIXTURE OF EXPERTS (MoE)

**Visual Architecture:**



# MIXTURE OF EXPERTS (MoE)

## MoE Architecture – Mathematical Formulation:

**Gating Function**

$$G(x) = \text{softmax}(W_g \cdot x + \varepsilon)$$

Where:

- $W_g$ = gating weight matrix
- $x$ = input vector
- $\varepsilon$ = small noise added during training

**Final Output**

$$y = \Sigma\, G_i(x) \cdot E_i(x)$$

Where:

- $G_i(x)$ = probability assigned to expert i
- Sum of all gate probabilities equals 1

**Expert Outputs**

Each expert computes:

$E_i(x)$

for i = 1 … N experts.

**Sparse MoE (Top-K Routing)**

Instead of using all experts:

I. Select top K experts with highest gate scores
II. Set other gate probabilities to zero
III. Renormalize the remaining probabilities

Final output:

$$y = \Sigma\, (i \in \text{Top-K})\, G_i(x) \cdot E_i(x)$$

Typical value: **K = 2**

**Benefits**

- Only a few experts are active per input
- Large model capacity
- Efficient computation
- Scales to very large parameter counts

# MIXTURE OF EXPERTS (MoE)

## MoE Numerical Example

Input vector:
x = [1.0, 0.5]

**Expert Outputs**

Expert 1
E1(x) = 0.8x1 + 0.3x2
= 0.8(1.0) + 0.3(0.5)
= 0.95

Expert 2
E2(x) = 0.4x1 + 0.9x2
= 0.4(1.0) + 0.9(0.5)
= 0.85

Expert 3
E3(x) = 0.2x1 + 0.5x2
= 0.2(1.0) + 0.5(0.5)
= 0.45

**Gating Network**

Weight matrix:
W_g = [[0.5, 0.3], (weights for expert1)
[0.2, 0.7], (weights for expert2)
[0.1, 0.4]] (weights for expert3)

Gate logits: Wg · x
[0.5×1.0+0.3×0.5, 0.2×1.0+0.7×0.5, 0.1×1.0+0.4×0.5]

= [0.65, 0.55, 0.30]

**Gate Probabilities**

softmax([0.65, 0.55, 0.30])
≈ [0.38, 0.34, 0.28]

**Final Output**

y = 0.38(0.95) + 0.34(0.85) + 0.28(0.45)
y = 0.361 + 0.289 + 0.126
y = **0.776**

# MIXTURE OF EXPERTS (MoE)

**Sparse MoE Example (Top-K = 2):**

Gate logits:[0.65, 0.55, 0.30]
Top-2 experts: Expert1 and Expert2

**Renormalized Gate Probabilities**
G1 = 0.65 / (0.65 + 0.55) = 0.542
G2 = 0.55 / (0.65 + 0.55) = 0.458
Expert3 probability = 0

**Final Output**
y = 0.542(0.95) + 0.458(0.85)
y = 0.515 + 0.389
y = **0.904**

Computational Savings:
Dense: 3 expert computations (100% compute)
Sparse: 2 expert computations (67% compute)
With 32 experts, K=2: only 6.25% compute!

| Method | Experts Used | Compute |
|---|---|---|
| Dense MoE | 3/3 | 100% |
| Sparse MoE | 2/3 | 67% |

Example with 32 experts:
Top-2 routing uses only **6.25% of experts per token**.

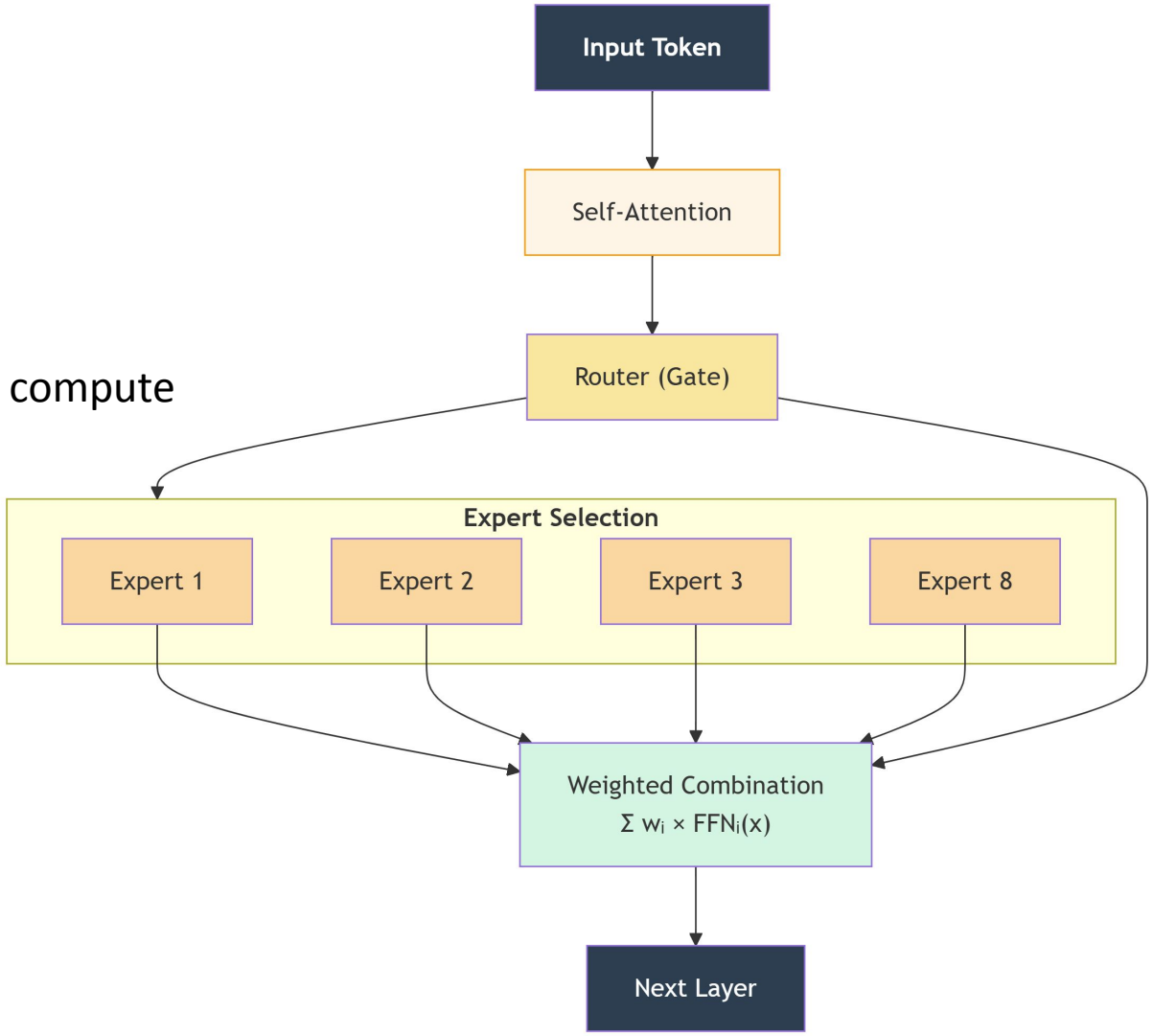
# MIXTURE OF EXPERTS (MoE)

## MoE in Large Language Models

MoE is commonly used in large transformer architectures.

**Key Characteristics**

- Each token routed to 1–2 experts
- Different tokens activate different experts
- Experts learn specialized representations
- Enables trillion-parameter models with sparse compute



# MIXTURE OF EXPERTS (MoE)

**MoE Numerical Example in LLM**

MoE is commonly used in large transformer architectures.

Input sentence: "The cat sat on the mat"

Token 1: "The" → Gate probabilities: [0.1, 0.7, 0.1, 0.1] → Expert2
Token 2: "cat" → Gate probabilities: [0.8, 0.1, 0.05, 0.05] → Expert1
Token 3: "sat" → Gate probabilities: [0.2, 0.2, 0.5, 0.1] → Expert3
Token 4: "on" → Gate probabilities: [0.15, 0.15, 0.6, 0.1] → Expert3
Token 5: "the" → Gate probabilities: [0.1, 0.7, 0.1, 0.1] → Expert2
Token 6: "mat" → Gate probabilities: [0.05, 0.1, 0.8, 0.05] → Expert3

Expert Specialization:
- Expert1: Nouns (cat, dog, house)
- Expert2: Articles/determiners (the, a, an)
- Expert3: Verbs/actions (sat, ran, jumped)
- Expert4: Prepositions (on, in, under) - lightly used

To prevent all tokens going to same expert:

$$L_balance = \alpha \cdot \Sigma (f_i \times P_i)$$

where:
f_i = fraction of tokens routed to expert i
P_i = average gate probability for expert i
α = balancing coefficient (typically 0.01)

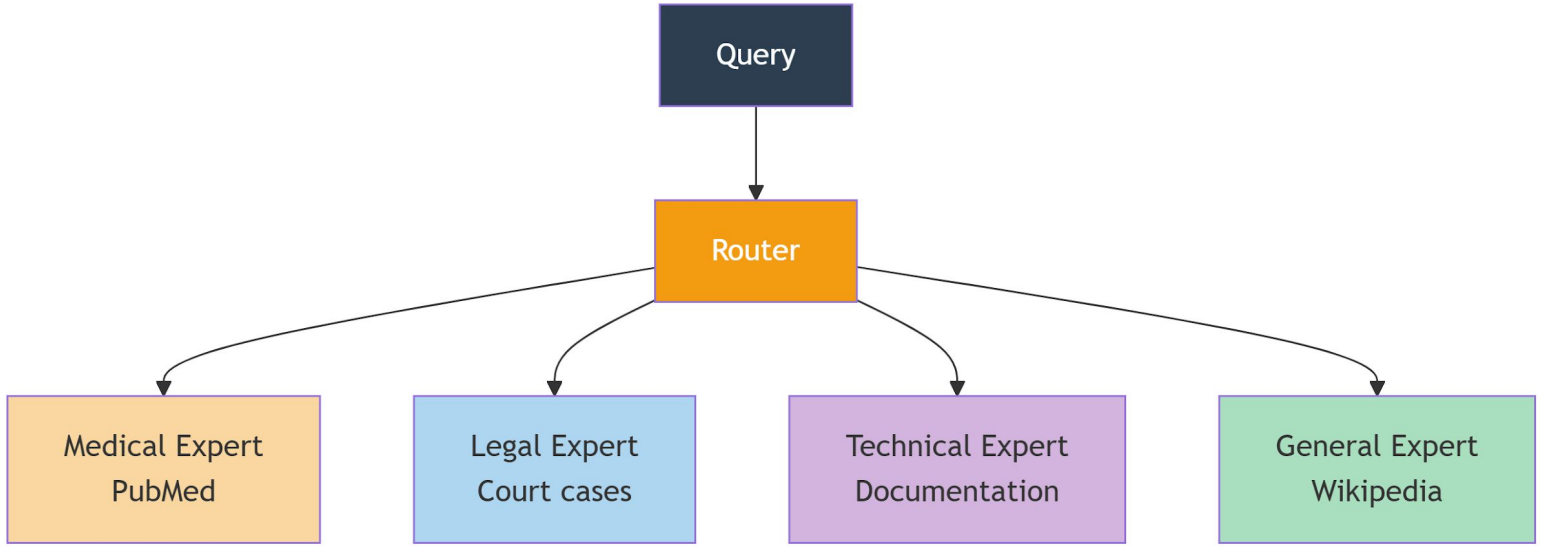
If f_i deviates from 1/N, loss increases!

# MIXTURE OF EXPERTS (MoE)

## MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

### 1. Multi-Domain Retrieval:



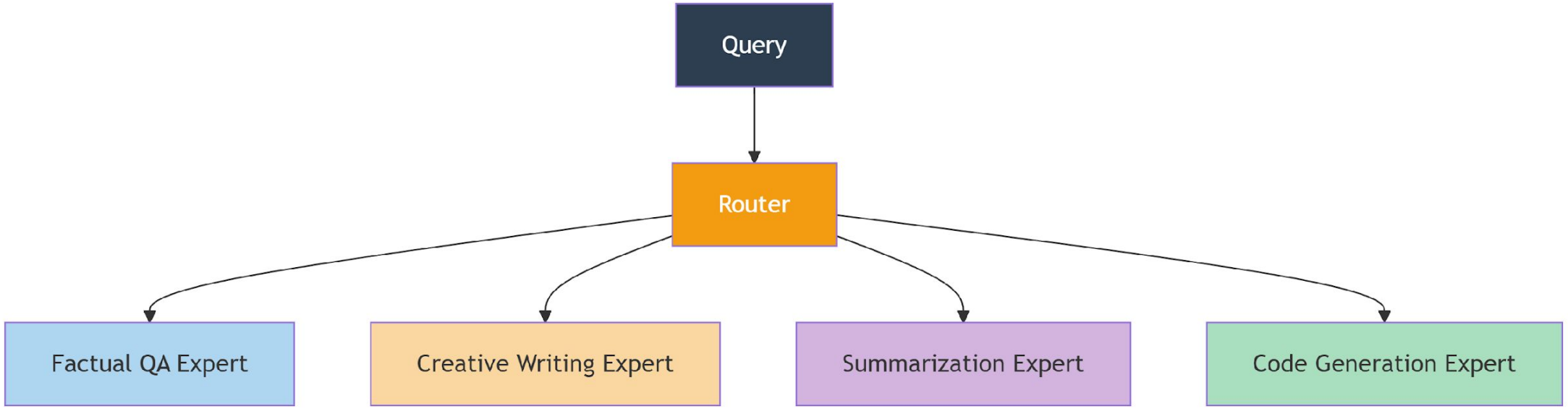

Query: "What are the side effects of Metformin?"
→ Routed to Medical Expert (90%), General Expert (10%)

# MIXTURE OF EXPERTS (MoE)

## MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

**2. Multi-Query Type Generation:**



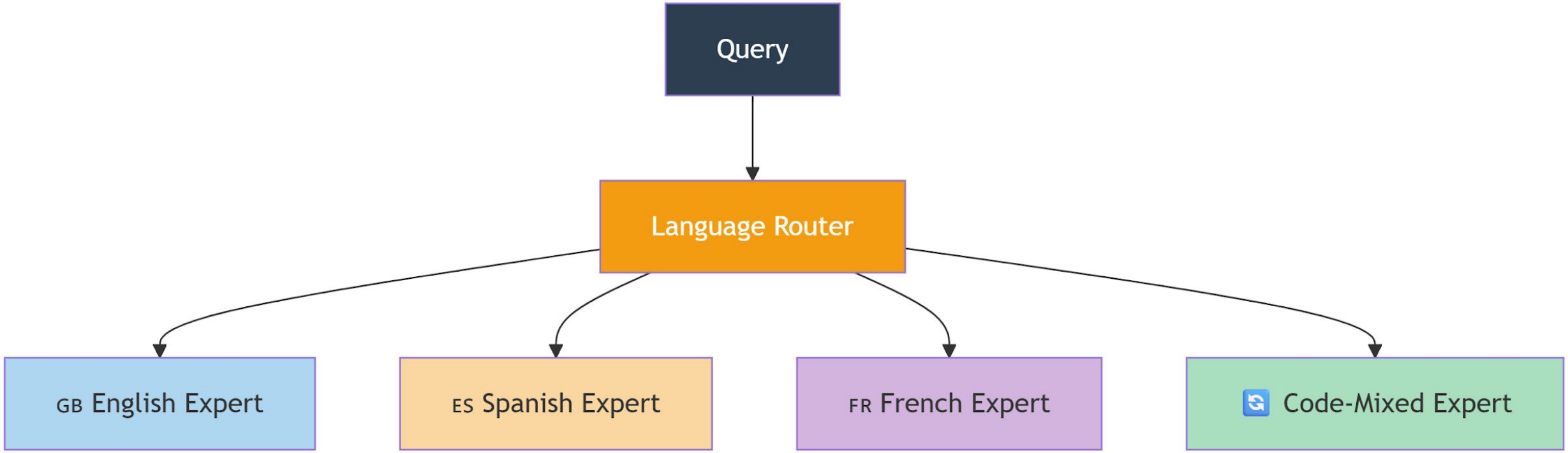

Query: "Write a poem about AI"
→ Routed to Creative Expert (95%), Factual Expert (5%)

# MIXTURE OF EXPERTS (MoE)

## MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

**3. Multi-Language Support:**



## MIXTURE OF EXPERTS (MoE)

**MoE Implementation Example:**

Example PyTorch implementation:

```
class MoELayer(nn.Module):

    def __init__(self, input_dim, output_dim, num_experts=8,
k=2):
        super().__init__()

        self.num_experts = num_experts
        self.k = k

        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.Linear(input_dim, input_dim * 4),
                nn.GELU(),
                nn.Linear(input_dim * 4, output_dim)
            )
            for _ in range(num_experts)
        ])

        self.gate = nn.Linear(input_dim, num_experts)
```

Forward pass:

1. Compute gate logits
2. Select top-k experts
3. Compute expert outputs
4. Combine outputs using gate probabilities

# MIXTURE OF EXPERTS (MoE)

**MoE Models in Production:**

| Model | Total Parameters | Active Experts | Use Case |
|---|---|---|---|
| Mixtral 8x7B | 47B | 2 of 8 | General LLM |
| Switch Transformer | 1.6T | 1 of 2048 | Large NLP models |
| GLaM | 1.2T | 2 of 64 | Efficient training |
| Qwen-MoE | 14B | 2 of 8 | Multilingual models |

**Mixtral 8x7B Example**
Architecture:
8 experts × 7B parameters
Total parameters ≈ 47B
Active experts per token = 2
Effective compute:
2 × 7B = **14B parameters**

Speed Comparison

| Model | Speed |
|---|---|
| Dense Llama-2 70B | 100 tokens/sec |
| Mixtral 8x7B | 180 tokens/sec |
| Mixtral 8x7B (4-bit) | 320 tokens/sec |

# MIXTURE OF EXPERTS (MoE)

## MoE Design Pattern for RAG

Conceptual architecture:

1. Query router selects domain experts
2. Domain-specific retrievers fetch documents
3. MoE generator produces the response

Steps:

1. Route query to top domains
2. Retrieve documents from selected domains
3. Weight documents by domain relevance
4. Generate answer using MoE generator

# Practice Problems

**Problem 1: LoRA Parameter Calculation**
Given a layer with input_dim=1024, output_dim=1024, calculate:
Original parameter count
LoRA parameters with r=16
Percentage of trainable parameters

**Problem 2: Quantization Error**
FP32 weights: [0.876, -1.234, 2.567, -3.145, 0.432]
Quantize to INT8 (symmetric) and calculate MSE error.

**Problem 3: MoE Routing**
Input x = [0.2, 0.8]
Gate weights W_g = [[0.3,0.7], [0.6,0.4], [0.2,0.5], [0.1,0.9]]
Expert outputs: [0.5, 0.8, 0.3, 0.6]
Calculate:
Gate logits and probabilities
Sparse output with K=2
Compare with dense output

**Problem 4: Fine-Tuning Strategy**
Design a fine-tuning strategy for a legal document Q&A system with:
10,000 legal documents
1,000 query-answer pairs
Need high precision (95%+)
Must run on 8GB GPU

# GenerativeAI and Its Applications

## Practice Problems

**Problem 1: LoRA Parameter Calculation**

**Given:**

- Layer: input_dim = 1024, output_dim = 1024
- LoRA rank r = 16

**Step 1: Original Parameter Count**

Original weight matrix $W \in \mathbb{R}^{1024 \times 1024}$

Parameters = 1024 × 1024 = 1,048,576

**Step 2: LoRA Parameters**

LoRA uses two matrices: $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$

where d = output_dim = 1024, k = input_dim = 1024, r = 16

Matrix A parameters = r × k = 16 × 1024 = 16,384

Matrix B parameters = d × r = 1024 × 16 = 16,384

Total LoRA parameters = 16,384 + 16,384 = 32,768

**Step 3: Percentage of Trainable Parameters**

Percentage = (LoRA parameters / Original parameters) × 100

= (32,768 / 1,048,576) × 100

= 3.125%

- Original parameters: **1,048,576**
- LoRA parameters: **32,768**
- Trainable percentage: **3.125%**

**Problem 2: Quantization Error**

**Given:** FP32 weights = [0.876, -1.234, 2.567, -3.145, 0.432]

**Step 1: Find Scale Factor (Symmetric Quantization)**

Max absolute value = max(|-3.145|, |2.567|, ...) = 3.145

INT8 range: -127 to 127

Scale factor s = max_abs / 127 = 3.145 / 127 = 0.02476

**Step 2: Quantize to INT8**

Formula: x_int = round(x / s)

0.876 / 0.02476 = 35.38 → 35

-1.234 / 0.02476 = -49.84 → -50

2.567 / 0.02476 = 103.68 → 104

-3.145 / 0.02476 = -127.02 → -127

0.432 / 0.02476 = 17.45 → 17

Quantized INT8: [35, -50, 104, -127, 17]

## Practice Problems Problem 2: Quantization Error

### Step 3: Dequantize

Formula: x_dequant = x_int × s

35 × 0.02476 = 0.8666

-50 × 0.02476 = -1.2380

104 × 0.02476 = 2.5750

-127 × 0.02476 = -3.1445

17 × 0.02476 = 0.4209

Dequantized: [0.8666, -1.2380, 2.5750, -3.1445, 0.4209]

### Step 4: Calculate MSE (Mean Squared Error)

Squared errors:

$(0.876 - 0.8666)^2 = (0.0094)^2 = 0.000088$

$(-1.234 - (-1.2380))^2 = (0.004)^2 = 0.000016$

$(2.567 - 2.5750)^2 = (-0.008)^2 = 0.000064$

$(-3.145 - (-3.1445))^2 = (-0.0005)^2 = 0.00000025$

$(0.432 - 0.4209)^2 = (0.0111)^2 = 0.000123$

MSE = (0.000088 + 0.000016 + 0.000064 + 0.00000025 + 0.000123) / 5

= 0.00029125 / 5

= 0.00005825

**Answer:** MSE = **$5.825 \times 10^{-5}$**

## Practice Problems Problem 3: MoE Routing

**Given:**

- Input x = [0.2, 0.8]
- Gate weights W_g = [[0.3, 0.7], [0.6, 0.4], [0.2, 0.5], [0.1, 0.9]]
- Expert outputs: [0.5, 0.8, 0.3, 0.6]

**Step 1: Calculate Gate Logits**

$Logit_1 = 0.3\times0.2 + 0.7\times0.8 = 0.06 + 0.56 = 0.62$

$Logit_2 = 0.6\times0.2 + 0.4\times0.8 = 0.12 + 0.32 = 0.44$

$Logit_3 = 0.2\times0.2 + 0.5\times0.8 = 0.04 + 0.40 = 0.44$

$Logit_4 = 0.1\times0.2 + 0.9\times0.8 = 0.02 + 0.72 = 0.74$

Gate logits = [0.62, 0.44, 0.44, 0.74]

**Step 2: Calculate Gate Probabilities (Softmax)**

Denominator $= e^{0.62} + e^{0.44} + e^{0.44} + e^{0.74}$

$= 1.859 + 1.553 + 1.553 + 2.096 = 7.061$

$P_1 = 1.859 / 7.061 = 0.263$

$P_2 = 1.553 / 7.061 = 0.220$

$P_3 = 1.553 / 7.061 = 0.220$

$P_4 = 2.096 / 7.061 = 0.297$

Gate probabilities = [0.263, 0.220, 0.220, 0.297]

**Step 3: Dense Output (All Experts)**

Dense output $= \Sigma P_i \times E_i$

$= 0.263\times0.5 + 0.220\times0.8 + 0.220\times0.3 + 0.297\times0.6$

$= 0.1315 + 0.176 + 0.066 + 0.1782$

$= 0.5517$

## Practice Problems Problem 3: MoE Routing

**Step 4: Sparse Output with K=2**

Top-2 logits: indices 3 (0.74) and 0 (0.62)

Renormalize top-2 probabilities:

Sum_top = 0.297 + 0.263 = 0.560

$P_3$_sparse = 0.297 / 0.560 = 0.530

$P_0$_sparse = 0.263 / 0.560 = 0.470

$P_1$_sparse = $P_2$_sparse = 0

Sparse output = 0.470×0.5 + 0×0.8 + 0×0.3 + 0.530 ×0.6

= 0.235 + 0 + 0 + 0.318

= 0.553

**Comparison:**

- Dense output: **0.5517**
- Sparse output (K=2): **0.5530**
- Difference: **0.0013** (0.24%)

**Answer:** Sparse routing with K=2 achieves **99.76%** of dense output quality with **50% fewer expert computations!**

## Practice Problems

**Problem 4: Fine-Tuning Strategy for Legal Document Q&A**

### Requirements Analysis:

- 10,000 legal documents (for retrieval)
- 1,000 query-answer pairs (for fine-tuning)
- 95%+ precision needed
- 8GB GPU constraint

### Phase 1: Retriever Fine-Tuning

Model: all-MiniLM-L6-v2 (80MB) or legal-bert (340MB)

Method: Multiple Negatives Ranking Loss

Data: Create triplets from 1,000 QA pairs

- Query + positive answer + negative (other answers)

Hard Negatives: Use BM25 to find similar but irrelevant docs

Batch Size: 32 (fits in 8GB)

Epochs: 3-5

## Practice Problems

### Problem 4: Fine-Tuning Strategy for Legal Document Q&A

**Phase 2: Reranker Fine-Tuning (Critical for 95% precision)**
Model: cross-encoder/ms-marco-MiniLM-L-6-v2 (120MB)
Method: Binary cross-entropy
Data: 1,000 QA pairs × 10 negatives = 10,000 examples
- Label 1 for positive pairs
- Label 0 for negative pairs

Quantization: INT8 post-training (30MB)
Batch Size: 16
Epochs: 2-3
Expected Improvement: +10-15% precision@5

**Phase 3: Generator (Optional - if response format matters)**
Model: Llama-2-7B (FP16 = 14GB → too large)
Solution: Use 4-bit quantized version (3.5GB)
Method: LoRA (r=8) with PEFT
Trainable params: ~0.1% (saves memory)
Data: 1,000 QA pairs formatted as:
"### Instruction: {query}\n### Context: {retrieved_docs}\n### Response: {answer}"
Batch Size: 4 with gradient accumulation
Epochs: 2

| Component | Before Fine-tuning | After Fine-tuning | Improvement |
|---|---|---|---|
| Retriever Recall@10 | 82% | 89% | +7% |
| Reranker Precision@5 | 78% | 94% | +16% |
| End-to-End Accuracy | 75% | 92% | +17% |

# THANK YOU